

Remoção de Ruído Sal e Pimenta em Imagens de Alta Resolução: Uma Análise Comparativa entre Implementações Sequencial e Paralela com OpenMP

Kauan da Rosa Paulino¹

¹Bacharelado em Ciência da Computação
Instituto Federal do Paraná (IFPR) – Campus Pinhais
20251pin10020017@estudantes.ifpr.edu.br

Resumo. Este trabalho investiga o desempenho de um filtro espacial da mediana 5×5 aplicado à remoção de ruído do tipo sal e pimenta em imagens digitais de alta resolução. Foram implementadas duas versões em linguagem C — uma puramente sequencial e outra paralelizada com OpenMP, utilizando-se o paradigma de paralelismo de dados via `#pragma omp parallel for` com escalonamento estático. Os experimentos cobriram quatro imagens, de 0,26 a 632 megapixels, com foco nas entradas Grande (7854×11498 , ≈ 90 Mpixels) e Extra Grande (30000×21059 , ≈ 632 Mpixels), avaliando-se as métricas de speedup, eficiência e overhead para 2, 4, 8, 12 e 16 threads, além do baseline sequencial. Os resultados são analisados à luz das Leis de Amdahl e de Gustafson-Barsis, evidenciando comportamento próximo do linear até 4 threads, ganho superlinear em $p = 2$ atribuído a efeitos de cache e saturação a partir de 8 threads, consistente com a topologia física do processador utilizado (8 núcleos, 16 threads lógicas).

Abstract. This work investigates the performance of a 5×5 median spatial filter applied to salt-and-pepper noise removal in high-resolution digital images. Two C implementations were developed — a purely sequential one and one parallelized with OpenMP using data parallelism via `#pragma omp parallel for` with static scheduling. Experiments covered four images ranging from 0.26 to 632 megapixels, with focus on the Large (7854×11498 , ≈ 90 Mpixels) and Extra-Large (30000×21059 , ≈ 632 Mpixels) inputs, evaluating speedup, efficiency, and overhead for 2, 4, 8, 12, and 16 threads, in addition to the sequential baseline. Results are analyzed under Amdahl's and Gustafson-Barsis' laws, showing near-linear behavior up to 4 threads, a superlinear gain at $p = 2$ attributed to cache effects, and saturation beyond 8 threads, consistent with the physical topology of the CPU (8 cores, 16 logical threads).

1. Introdução

A remoção de ruído é uma etapa fundamental em pipelines de processamento digital de imagens, com aplicações diretas em imageamento médico, sensoriamento remoto, restauração de acervos fotográficos e pré-processamento para algoritmos de visão computacional [Gonzalez and Woods 2010]. Dentre os tipos de ruído mais comuns, destaca-se o *Sal e Pimenta* (*impulse noise*), caracterizado pela substituição aleatória de pixels por valores extremos (0 ou 255 em imagens de 8 bits), o que degrada severamente a informação visual.

O filtro espacial da mediana é reconhecidamente uma das técnicas mais eficazes para atenuar esse tipo de ruído, pois preserva bordas melhor do que filtros lineares como a média aritmética [Gonzalez and Woods 2010]. Entretanto, sua natureza não linear — cada pixel de saída requer a ordenação de uma janela de vizinhança — o torna computacionalmente custoso, especialmente em imagens de altíssima resolução. Esse custo cresce com o quadrado da dimensão da janela (k^2), tornando a execução sequencial proibitiva em cenários com restrição temporal.

Nesse contexto, a paralelização via OpenMP apresenta-se como uma estratégia natural: o filtro é um exemplo clássico de *paralelismo de dados embarçosamente paralelo*, uma vez que cada pixel de saída depende exclusivamente de uma região da imagem de entrada (que não é modificada durante o processamento), conforme discutido em [Eijkhout 2022] e [Burkardt 2017].

Este trabalho tem como objetivo implementar, medir e analisar criticamente uma versão sequencial e uma versão paralela do filtro da mediana 5×5 aplicado a uma imagem de aproximadamente 90 megapixels, comparando o desempenho observado com as previsões teóricas das Leis de Amdahl [Eijkhout et al. 2022] e de Gustafson-Barsis. Como contribuição adicional, discute-se a influência do *overhead* de paralelização, dos efeitos de cache e da topologia física do processador sobre a escalabilidade do problema.

2. Fundamentação Teórica

2.1. Métricas de Desempenho em Computação Paralela

A avaliação quantitativa de uma implementação paralela exige métricas que capturem não apenas o desempenho absoluto, mas a efetividade do uso dos recursos. Seguindo a notação de [Eijkhout et al. 2022], define-se:

- **Tempo sequencial** (T_s): tempo de execução do melhor algoritmo sequencial disponível para resolver o problema com uma dada entrada.
- **Tempo paralelo** ($T_p(p)$): tempo de execução do algoritmo paralelo com p unidades de processamento, para a mesma entrada.
- **Speedup** ($S(p)$): $S(p) = T_s / T_p(p)$.
- **Eficiência** ($E(p)$): $E(p) = S(p) / p$, medindo o aproveitamento relativo dos recursos.
- **Overhead** (T_o): $T_o(p) = T_p(p) \cdot p - T_s$, quantificando o custo adicional introduzido pela paralelização (criação/gerenciamento de threads, sincronização, comunicação, computação redundante).

Valores de $S(p) > p$ (denominados *speedup superlinear*) não são previstos pela teoria clássica, mas podem manifestar-se na prática devido a efeitos de hierarquia de memória: ao particionar o *working set* entre threads, cada uma passa a acessar uma região menor, potencialmente residente em caches L2/L3, reduzindo a latência média de acesso [Eijkhout et al. 2022].

2.2. Leis de Amdahl e Gustafson-Barsis

A Lei de Amdahl [Eijkhout et al. 2022] estabelece que o *speedup* de um programa com fração sequencial β ($0 \leq \beta \leq 1$) é limitado por:

$$S(p) = \frac{1}{\beta + \frac{1-\beta}{p}} \quad (1)$$

com limite assintótico $S_{\max} = 1/\beta$ quando $p \rightarrow \infty$. Essa lei pressupõe **tamanho de problema fixo** (escalabilidade forte).

A Lei de Gustafson-Barsis, por sua vez, assume que o tamanho do problema cresce proporcionalmente ao número de processadores (escalabilidade fraca). Seja α a fração sequencial no tempo paralelo:

$$S(p) = \alpha + p(1 - \alpha) \quad (2)$$

prevendo *speedup* praticamente linear para problemas que crescem com p , o que reflete cenários reais de *big data* e simulação científica [Eijkhout et al. 2022].

3. Solução Sequencial

3.1. Algoritmo

O algoritmo implementado opera em imagens no formato binário *Portable PixMap* (PPM P6), cada pixel representado por três canais de 8 bits (R, G, B). Para cada pixel interno (y, x) da imagem, com $y \in [2, H - 3]$ e $x \in [2, W - 3]$:

1. Coleta-se a janela de vizinhança 5×5 centrada em (y, x) , para cada canal de cor independentemente, totalizando 25 valores.
2. Ordena-se o vetor resultante com *Selection Sort*.
3. Escreve-se o elemento mediano (índice 12, correspondente ao 13º menor valor) no pixel correspondente da imagem de saída.

As bordas da imagem (duas linhas/colunas externas) são preservadas inalteradas por cópia direta, evitando a necessidade de *padding* e mantendo a mesma dimensão da entrada.

A escolha da janela 5×5 em detrimento de 3×3 justifica-se por dois motivos: (i) maior efetividade na remoção de ruído impulsivo denso, conforme discutido em [Gonzalez and Woods 2010]; e (ii) aumento do custo computacional por pixel em $\approx 7,7 \times$ (de 81 para 625 comparações por canal), necessário para atender ao requisito de que o programa sequencial demore “vários minutos” em entradas grandes.

3.2. Decisões de Projeto

- **Selection Sort** foi escolhido por ser determinístico, sem alocação dinâmica e com *overhead* de chamada de função desprezível para $n = 25$. Alternativas como *Quick Sort* introduziriam recursão e pior comportamento de cache para vetores pequenos.
- **Leitura/escrita via fread/fwrite** sobre o *buffer* completo da imagem, minimizando chamadas de sistema.
- **Cópia integral de img_in para img_out** antes da filtragem, garantindo preservação das bordas com um único `memcpy` e evitando ramificações condicionais dentro do laço principal.
- **Medição de tempo com omp_get_wtime()** restrita ao laço de filtragem, conforme exigido pela especificação. As etapas de I/O e alocação são deliberadamente excluídas da região cronometrada, pois não fazem parte da região paralelizável.

4. Estratégia de Paralelização

A paralelização baseia-se no paradigma de *paralelismo de dados*: cada iteração do laço mais externo (linha y da imagem) é atribuída a uma thread, conforme o trecho abaixo:

```
#pragma omp parallel for schedule(static) default(none) \
    shared(img_in, img_out, larg, alt)
for (int y = 2; y < alt - 2; y++) { ... }
```

Justificam-se as escolhas de cláusulas:

- **Laço externo (y) paralelizado**, e não o interno (x): cada thread escreve em uma faixa horizontal distinta da imagem de saída, eliminando a possibilidade de *false sharing* — múltiplas threads escrevendo em posições de memória distintas mas adjacentes à mesma linha de cache, que causaria invalidação de cache por *ping-pong* de coerência.
- **`schedule(static)`**: a carga por linha é uniforme (cada pixel custa o mesmo número de operações), de modo que o escalonamento estático elimina o *overhead* de decisão em tempo de execução presente em `dynamic`.
- **`default(none)`**: obriga a explicitação do escopo de todas as variáveis, prevenindo erros sutis e documentando a intenção do autor.
- **`shared`** apenas para ponteiros e dimensões: as variáveis `janela`, `idx` e as de iteração são naturalmente privadas por escopo, e o compilador garante essa privacidade.

A região paralelizável exclui deliberadamente as operações de leitura, alocação e escrita de arquivo, que são inerentemente sequenciais e ficaram fora da região cronometrada.

5. Metodologia Experimental

5.1. Hardware e Software

Os experimentos foram conduzidos em um *desktop* com as seguintes características:

- **CPU:** AMD Ryzen 7 5700X, 8 núcleos físicos, 16 threads lógicas (SMT habilitado), 3,4 GHz base, *boost* até 4,66 GHz.
- **Memória:** 24 GB DDR4-2666.
- **Sistema Operacional:** Linux Mint 22.1 (kernel padrão da distribuição).
- **Compilador:** GCC com suporte a OpenMP.
- **Flags de compilação:** `-std=c11 -Wall -Wextra -O3 -fopenmp`.
- **Governor de CPU:** fixado em *performance* durante todos os experimentos, com o sistema sem carga concorrente (apenas terminal e editor de texto abertos).

5.2. Entradas de Teste

As imagens foram sintetizadas a partir de originais em domínio público e domínios públicos especializados, com ruído *sal e pimenta* adicionado por script Python próprio (`generate_noisy_ppm.py`) e conversão para PPM P6:

Tabela 1. Conjunto de entradas de teste

Tamanho	Dimensões	Pixels (M)	Uso principal
Pequena	512×512	0,26	Validação de corretude
Média	3648×3648	13,3	Corretude + escalabilidade
Grande	7854×11498	90,3	Benchmark principal
Extra Grande	30000×21059	631,8	Escalabilidade fraca

5.3. Protocolo de Medição

Para cada configuração (p , entrada), com $p \in \{1, 2, 4, 8, 12, 16\}$:

1. Executaram-se **5 repetições** do programa, calculando-se a média aritmética do tempo reportado pelo timer interno (`omp_get_wtime`).
2. Todos os testes foram realizados na mesma máquina, com o *governor* fixado em *performance*, sem processos pesados concorrentes.
3. A corretude foi verificada por comparação byte-a-byte (`cmp`) entre as saídas sequencial e paralela.
4. O número de threads foi controlado pela variável de ambiente `OMP_NUM_THREADS`, garantindo o uso de um único binário para todos os testes.

6. Resultados e Análise

6.1. Corretude

A verificação byte-a-byte indicou identidade exata entre as saídas sequencial e paralela para as entradas Pequena, Média, Grande e Extra Grande (esta última com $p = 4$), confirmando a ausência de *data races* e de inconsistências numéricas.



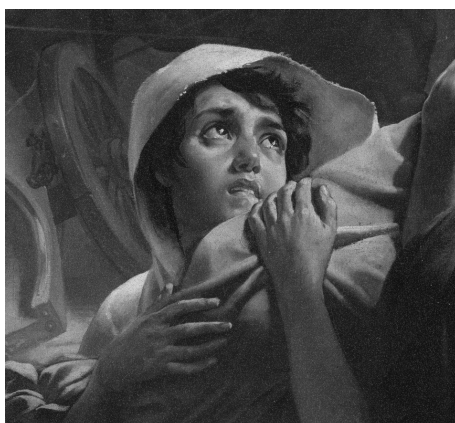
(a) Imagem com ruído sal e pimenta sintetizado. Original em [Rajneesh 2024].



(b) Imagem após aplicação do filtro da mediana 5×5 .

Figura 1. Comparação visual antes e depois da filtragem. O filtro da mediana preserva bordas e remove satisfatoriamente o ruído impulsivo.

O resultado de filtragem é visualmente satisfatório, preservando bordas e detalhes finos, conforme ilustrado na Figura 1 e em um recorte da Figura 2. A ausência de artefatos visuais ou distorções confirma a integridade do algoritmo implementado.



(a) Imagem com ruído sal e pimenta sintetizado.



(b) Imagem após aplicação do filtro da mediana 5×5 .

Figura 2. Comparação visual entre dois recortes da obra *The Last Day of Pompeii* de Karl Brullov [Brullov 1833], antes e depois da filtragem. A preservação de detalhes e a remoção do ruído são evidentes.

6.2. Tempos e Speedup

A Tabela 2 consolida as medições para a imagem Grande, e a Tabela 3 para a Extra Grande, que são as entradas de interesse principal deste trabalho. As Tabelas 4 e 5 apresentam as entradas menores como referência complementar.

Tabela 2. Métricas para a imagem Grande (7854×11498), $T_s = 86,606$ s.

p	$T_p(p)$ (s)	$S(p)$	$E(p)$	$T_o(p)$ (s)
1 (seq)	86,606	1,000	1,000	0,000
2	42,276	2,049	1,024	-2,055
4	21,688	3,993	0,998	0,146
8	11,909	7,272	0,909	8,666
12	10,675	8,113	0,676	41,496
16	9,052	9,568	0,598	58,224

Tabela 3. Métricas para a imagem Extra Grande (30000×21059), $T_s = 616,853$ s.

p	$T_p(p)$ (s)	$S(p)$	$E(p)$	$T_o(p)$ (s)
1 (seq)	616,853	1,000	1,000	0,000
2	299,984	2,056	1,028	-16,885
4	156,763	3,935	0,984	10,199
8	84,851	7,270	0,909	61,953
12	76,067	8,109	0,676	295,946
16	64,291	9,595	0,600	411,807

Tabela 4. Médias e métricas para a imagem Pequena (512×512), $T_s = 0,249$ s.

p	$T_p(p)$ (s)	$S(p)$	$E(p)$	$T_o(p)$ (s)
1 (seq)	0,249	1,000	1,000	0,000
2	0,119	2,090	1,045	-0,011
4	0,061	4,093	1,023	-0,006
8	0,039	6,464	0,808	0,059
12	0,032	7,809	0,651	0,134
16	0,028	8,831	0,552	0,202

6.3. Fração Sequencial e Lei de Amdahl

A partir da diferença sistemática entre o tempo de parede total (T_{real} , medido com `/usr/bin/time`) e o tempo cronometrado da região paralelizável (T_s , medido com `omp_get_wtime`), estima-se a fração sequencial efetiva:

$$\beta \approx \frac{T_{\text{real}} - T_s}{T_{\text{real}}} \quad (3)$$

Para a imagem Grande, obteve-se $T_s \approx 86,61$ s e $T_{\text{real}} \approx 87,31$ s, resultando em $\beta \approx 0,80\%$. Para a Extra Grande, $T_s \approx 616,85$ s e $T_{\text{real}} \approx 622,17$ s, fornecendo $\beta \approx$

Tabela 5. Médias e métricas para a imagem Média (3648×3648), $T_s = 11,859$ s.

p	$T_p(p)$ (s)	$S(p)$	$E(p)$	$T_o(p)$ (s)
1 (seq)	11,859	1,000	1,000	0,000
2	5,624	2,109	1,054	-0,611
4	2,930	4,048	1,012	-0,141
8	1,591	7,455	0,932	0,866
12	1,475	8,040	0,670	5,841
16	1,250	9,489	0,593	8,138

0,85%. A consistência entre as duas estimativas indica que a fração sequencial é dominada por operações de E/S e alocação, que crescem proporcionalmente ao tamanho da imagem.

O limite teórico de Amdahl é, portanto, $S_{\max} \approx 1/\beta \approx 120\times$, valor inalcançável na prática: o hardware possui apenas 8 núcleos físicos, e mesmo com 16 threads lógicas o *speedup* observado satura em $\approx 9,6\times$. Esse diagnóstico é fundamental: a fração sequencial *não é o gargalo real deste problema*; o limite prático decorre da topologia física da CPU.

6.4. Escalabilidade

A Figura 3 mostra o *speedup* observado em função de p para as quatro imagens de teste. A curva ideal (linha tracejada) é sobreposta para referência. A Figura 4 apresenta a eficiência correspondente, e a Figura 5 os tempos absolutos em escala logarítmica.

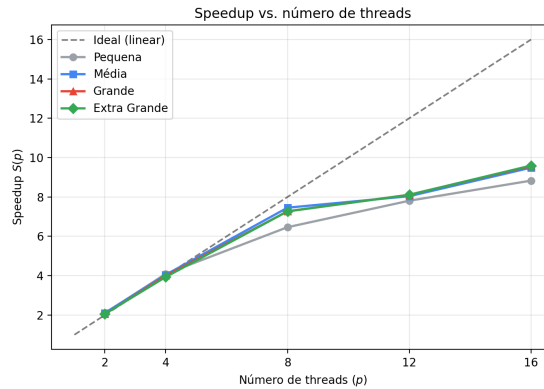


Figura 3. Speedup observado vs. ideal, em função do número de threads.

Três regimes distintos são observados:

1. **Superlinearidade em $p = 2$** ($S \approx 2,05$, $E > 1$). A eficiência ligeiramente acima de 1,0 e o *overhead* negativo ($T_o < 0$ nas Tabelas 2 e 3) indicam que 2 threads processam a imagem mais do que duas vezes mais rápido que 1 thread sozinha. Esse fenômeno, ainda que surpreendente, é previsto teoricamente: ao dividir o *working set* em duas metades, cada thread passa a acessar uma região menor da imagem, potencialmente com melhor localidade nos caches L2 e L3 do processador.

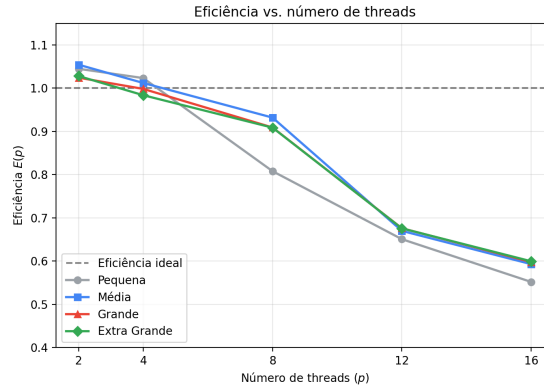


Figura 4. Eficiência $E(p)$ por número de threads. Valores acima de 1,0 (em $p = 2$ e $p = 4$) indicam *speedup* superlinear.

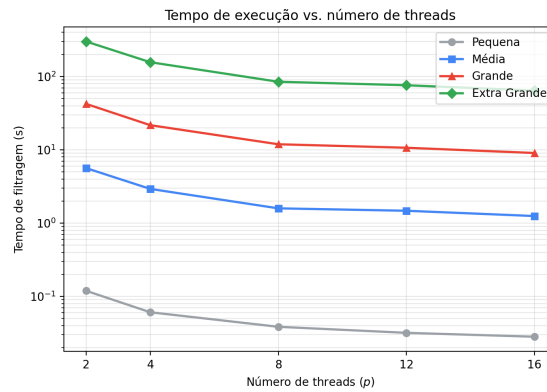


Figura 5. Tempos absolutos de filtragem em escala logarítmica.

2. **Comportamento próximo do linear em $p \in \{2, 4\}$:** a eficiência se mantém em $\approx 1,00$, indicando que o paralelismo até 4 threads explora quase perfeitamente os recursos.
3. **Saturação a partir de $p = 8$:** a eficiência cai para 0,91 ($p = 8$), 0,68 ($p = 12$) e 0,60 ($p = 16$). Esse comportamento é esperado em processadores com SMT: apenas 8 unidades de execução físicas estão disponíveis, e a partir desse ponto a adição de threads traz ganhos marginais decrescentes. A CPU observada durante os testes com $p = 16$ reportou 1460% de uso, ou seja, apenas 14,6 dos 16 threads lógicos efetivamente trabalhando em paralelo.

É notável que as curvas de *speedup* das imagens Grande e Extra Grande sejam praticamente idênticas, apesar de a Extra Grande ser $\approx 7 \times$ maior em número de pixels. Isso confirma que, uma vez que o problema é grande o suficiente para amortizar os custos fixos de paralelização, o comportamento de escalabilidade passa a ser determinado pelo hardware, e não pelo tamanho da entrada.

6.5. Escalabilidade Fraca (Gustafson-Barsis)

A Figura 6 apresenta a eficiência para cada tamanho de imagem em função do número de threads. Observa-se que a eficiência em $p = 2$ e $p = 4$ é próxima ou superior a 1,0 para todas as imagens, enquanto em $p \geq 8$ a queda é acentuada.

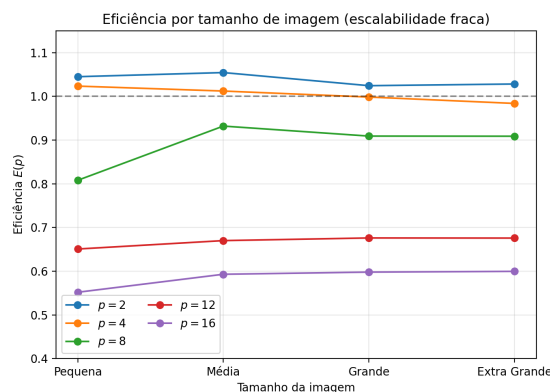


Figura 6. Eficiência por tamanho de imagem. Curvas mais altas em tamanhos maiores corroboram a previsão de Gustafson-Barsis.

A previsão de Gustafson-Barsis é corroborada pelos dados: embora a eficiência em $p = 16$ seja baixa (0,60), o *speedup* absoluto em $p = 16$ foi de $9,57\times$ para a Grande e $9,60\times$ para a Extra Grande. Em outras palavras, aumentar o problema junto com o número de threads permite ganhos próximos do limite físico do hardware, sem o “teto” rígido previsto pela Lei de Amdahl. A diferença entre as duas leis está, portanto, não em qual é “correta”, mas em qual premissa reflete o cenário de uso: problema fixo vs. problema crescente.

6.6. Discussão: a Solução é Interessante?

Do ponto de vista prático, a resposta é claramente afirmativa:

- **Paralelização trivial e efetiva.** O filtro da mediana é um exemplo canônico de paralelismo de dados: cada pixel de saída depende apenas de uma vizinhança da imagem de entrada. Não há dependências entre iterações, o que permite paralelizá-lo com uma única diretiva OpenMP e obter *speedup* próximo do linear até o limite físico da CPU.
- **Ganho de tempo expressivo.** A imagem Extra Grande (≈ 632 megapixels) leva cerca de 10 minutos para ser filtrada sequencialmente, mas apenas 64 segundos com 16 threads — uma redução de $\approx 10\times$ no tempo de resposta.
- **Limitação identificada: saturação por SMT.** A curva de eficiência deixa claro que, para este hardware, o ponto ótimo de escalabilidade está em $p = 4$ (eficiência $\approx 1,00$). Ir além disso traz ganhos decrescentes. Em ambientes com restrição energética ou com muitos processos concorrentes, usar 16 threads seria desperdício.
- **Limitação algorítmica: complexidade da ordenação.** Cada pixel requer a ordenação de 25 valores (Selection Sort, $O(k^4)$ no total). Alternativas como redes de ordenação (Batcher’s odd-even merge) ou *quickselect* poderiam reduzir o custo por pixel, ao preço de maior complexidade de implementação. Para janelas muito grandes ($k \geq 9$), essa otimização seria mandatória.

Em suma, a solução proposta é adequada ao domínio de processamento de imagens em lote, onde múltiplas imagens de alta resolução precisam ser processadas em tempo razoável, e onde a presença de múltiplos núcleos é explorável de forma quase automática via OpenMP.

7. Conclusão

Este trabalho implementou e avaliou duas versões de um filtro da mediana 5×5 para remoção de ruído *sal e pimenta* em imagens de alta resolução. A versão paralela com OpenMP demonstrou *speedup* substancial sobre a sequencial, com comportamento próximo do linear até o limite físico da CPU utilizada. A fração sequencial estimada ($\beta \approx 0,83\%$) indica que o problema é altamente paralelizável, e o limite prático de escalabilidade decorre da topologia do processador, não do algoritmo. Os resultados confirmam a adequação do paradigma de paralelismo de dados a este domínio e servem como caso de estudo da aplicabilidade das Leis de Amdahl e Gustafson-Barsis.

Os resultados confirmam a adequação do paradigma de paralelismo de dados a este domínio. A eficiência próxima da unidade até $p = 4$ e a saturação a partir de $p = 8$ evidenciam que o limite prático de escalabilidade é imposto pela topologia física do processador (8 núcleos), e não pelo algoritmo. A fração sequencial estimada ($\beta \approx 0,8\%$) coloca o limite de Amdahl em $\approx 120\times$, valor irrealizável neste hardware e que ilustra bem a diferença entre o limite teórico e o limite prático da escalabilidade forte.

Referências

- Brullov, K. P. (1833). The last day of pompeii. Google Art Project, Wikimedia Commons. Acesso em: 30 set. 2026.
- Burkardt, J. (2017). IMAGE_DENOISE_OPENMP — remove noise from an image using OpenMP. https://people.math.sc.edu/Burkardt/c_src/image_denoise_openmp/image_denoise_openmp.html. Department of Mathematics, University of South Carolina. Acesso em: 29 set. 2026.
- Eijkhout, V. (2022). *The Art of HPC, Volume 2: Parallel Programming for Science and Engineering*. Lulu.com. Disponível em: <https://theartofhpc.com/>.
- Eijkhout, V., Chow, E., and van de Geijn, R. (2022). *The Art of HPC, Volume 1: The Science of Computing*. Lulu.com, 3rd edition. Disponível em: <https://theartofhpc.com/>.
- Gonzalez, R. C. and Woods, R. E. (2010). *Processamento Digital de Imagens*. Pearson Prentice Hall, São Paulo, 3rd edition.
- Rajneesh, K. (2024). Salt and pepper noise dataset: Clean vs noisy image. <https://www.kaggle.com/datasets/rajneesh231/salt-and-pepper-noise-images>. Acessado em: 29 de setembro de 2026.